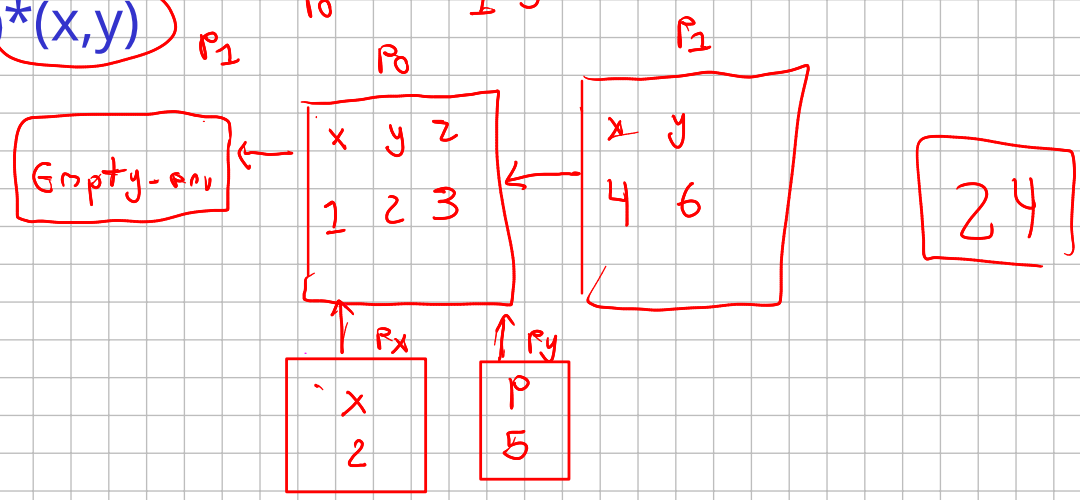
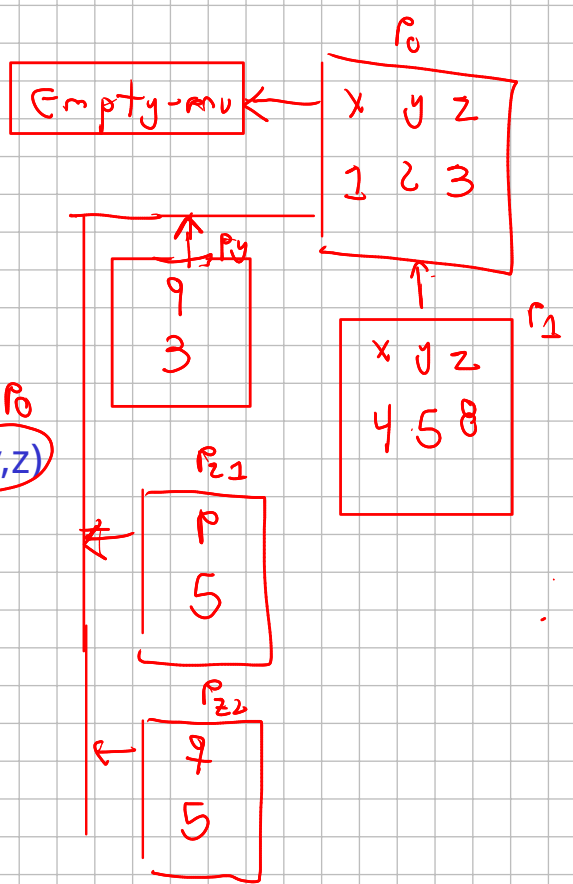


let x = 1 y = 2 z = 3 in let x = let x = ~~+(x,1)~~ in ~~*(x,2)~~
 y = let p = ~~+(y,z)~~ in ~~+(x,p)~~
 in ~~*(x,y)~~



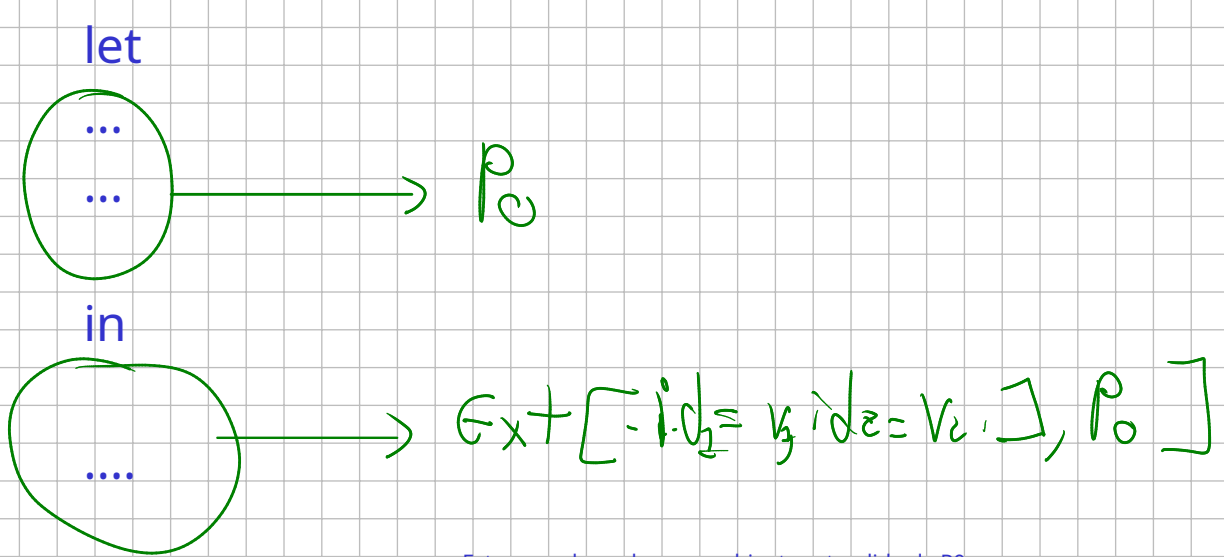
let
 x = 1
 y = 2
 z = 3
 in
 let
 x = if >(x,1) then +(x,y) else +(x,z)
 y = if let q = +(x,2) in >(q,10) then +(x,z) else +(y,z)
 z = let p = let q = +(y,z) in *(x,q) in +(p,3)
 in +(x,y,z)

No genera un ambiente



$+(4, 5, 8) = 17$

Suponga un ambiente inicial p0



Este se evalua sobre un ambiente extendido de P0

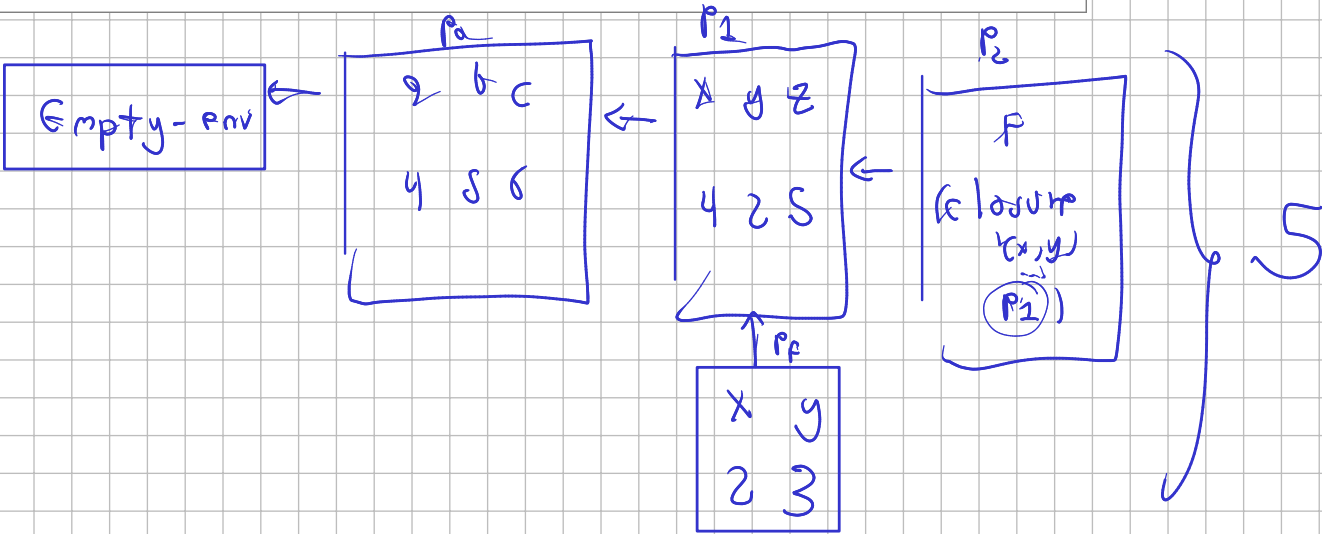
Clausura

- lista de argumentos
- El cuerpo
- El ambiente donde FUE CREADO EL PROCEDIMIENTO

```
-->proc (x,y) +(x,y)
#(struct:closure (x y) #(struct:prim-exp #(struct:sum-prim) (#(struct:var-exp x)
#(struct:var-exp y))) #(struct:ambiente-extendido (x y z) (4 2 5)
#(struct:ambiente-extendido (a b c) (4 5 6) #(struct:ambiente-vacio)))
```

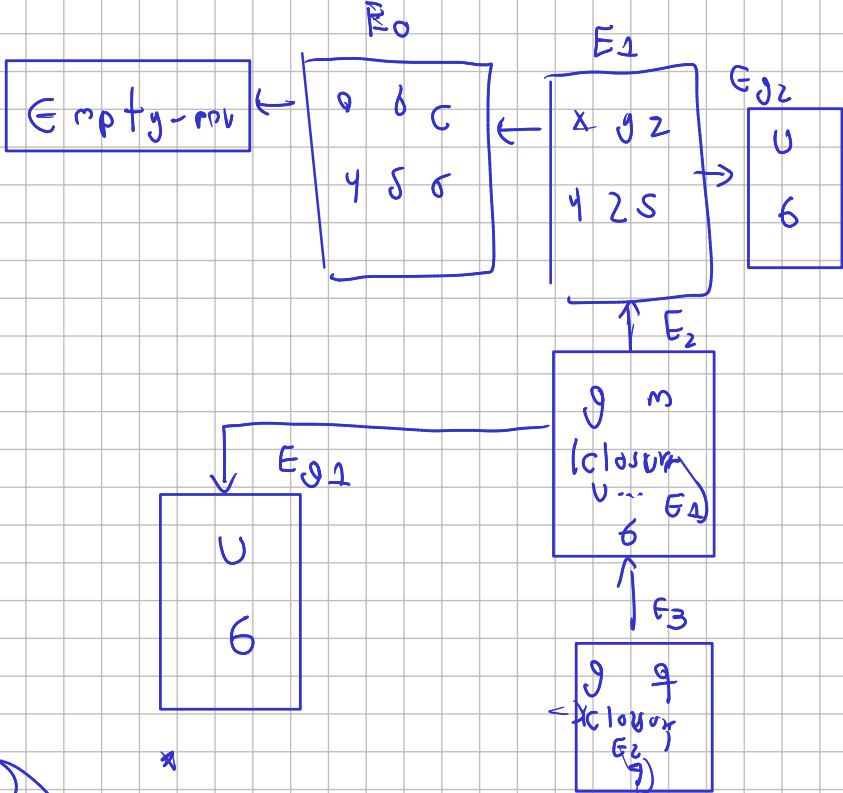
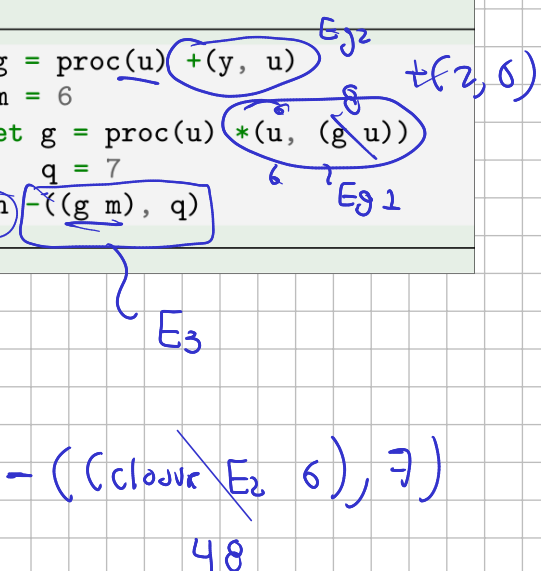
```
;; Ambiente inicial de prueba:
;; [x=4, y=2, z=5] -> [a=4, b=5, c=6] -> ∅ (vacío)
(define ambiente-inicial
  (ambiente-extendido '(x y z) '(4 2 5)
    (ambiente-extendido '(a b c) '(4 5 6)
      (ambiente-vacio))))
```

```
-->let f = proc(x,y) +(x,y) in (f 2 3)
5
```



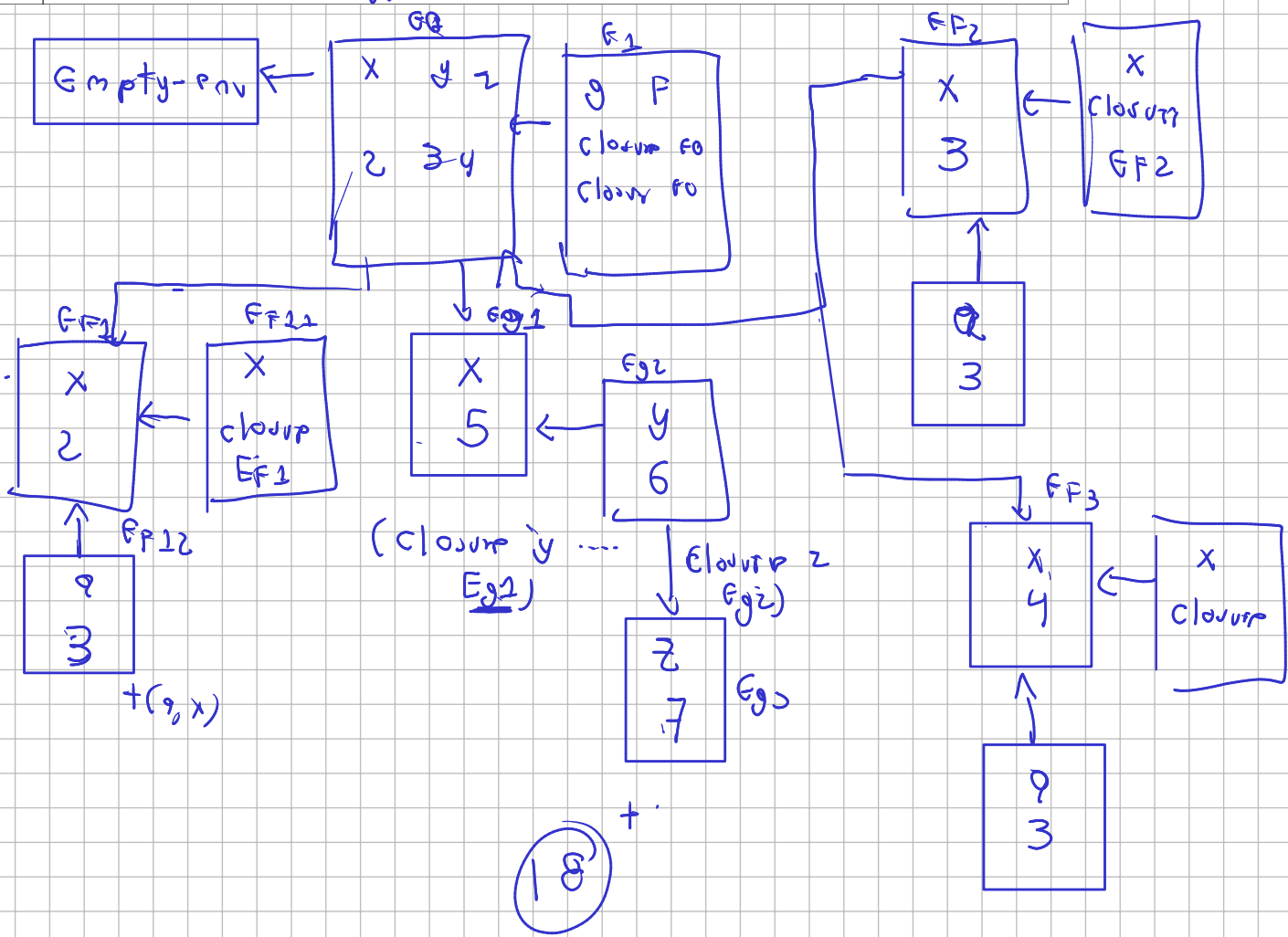
En env₀ (x=4, y=2, z=5)

```
let g = proc(u) +(y, u)
m = 6
in let g = proc(u) *(u, (g u))
q = 7
in ((g m), q)
```

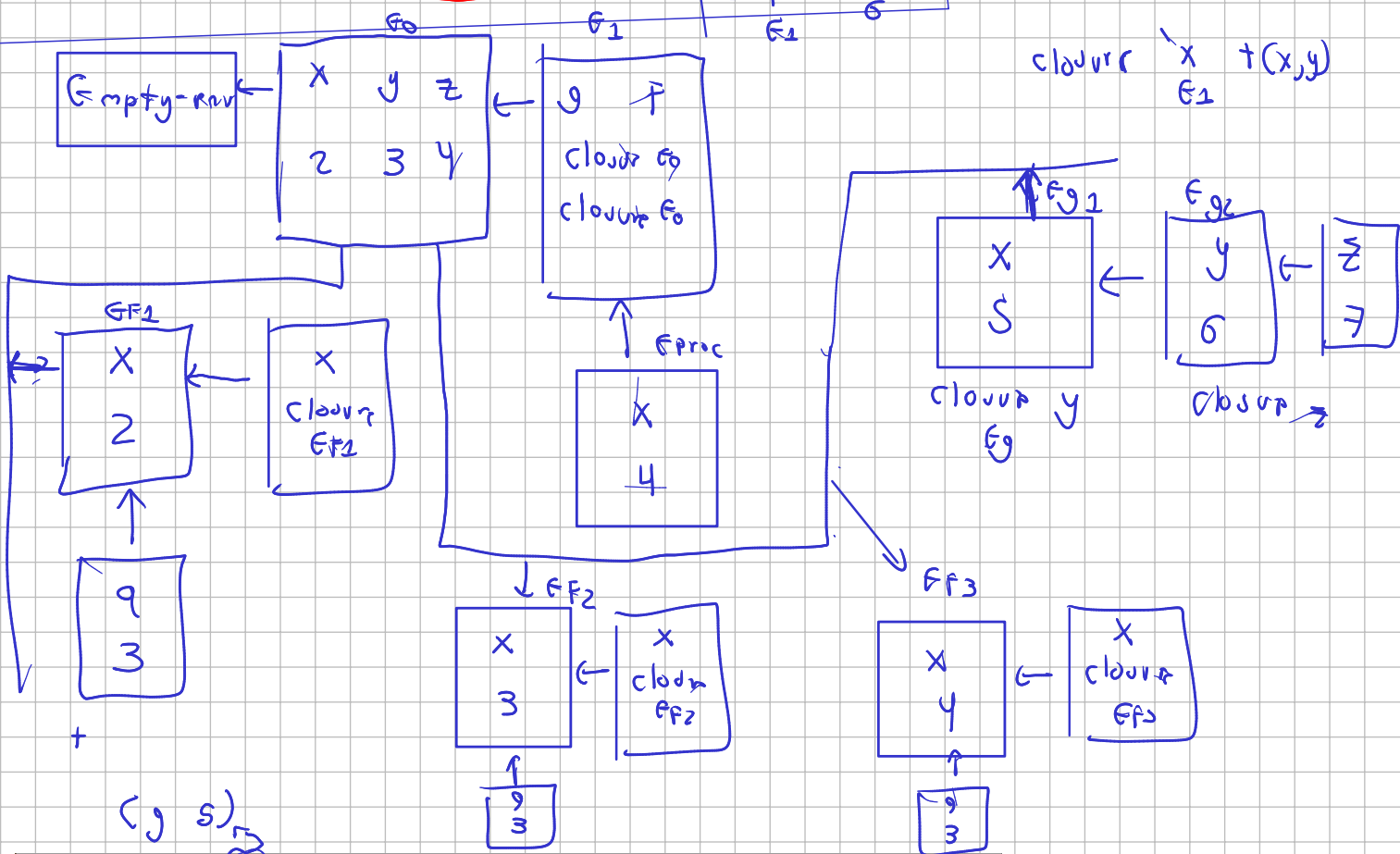


```
let
  g = proc(x) proc (y) proc (z) +(x,y,z)
  f = proc(x) let x = proc(a) +(a,x) in (x y)
in
  ( ( (g (f x)) (f y)) (f z))
```

18



```
let
  g = proc(x) proc (y) proc (z) +(x,y,z)
  f = proc(x) let x = proc(a) +(a,x) in (x y)
in
  +( (proc (x) +(x,y) z) (((g (f x)) (f y)) (f z)))
```



```
#(struct:closure (y) #(struct:proc-exp (z) #(struct:prim-exp #(struct:sum-prim)
#(struct:var-exp x) #(struct:var-exp y) #(struct:var-exp z))))
#(struct:ambiente-extendido (x) (5) #(struct:ambiente-extendido (x y z) (2 3 4)) Empty-env
```

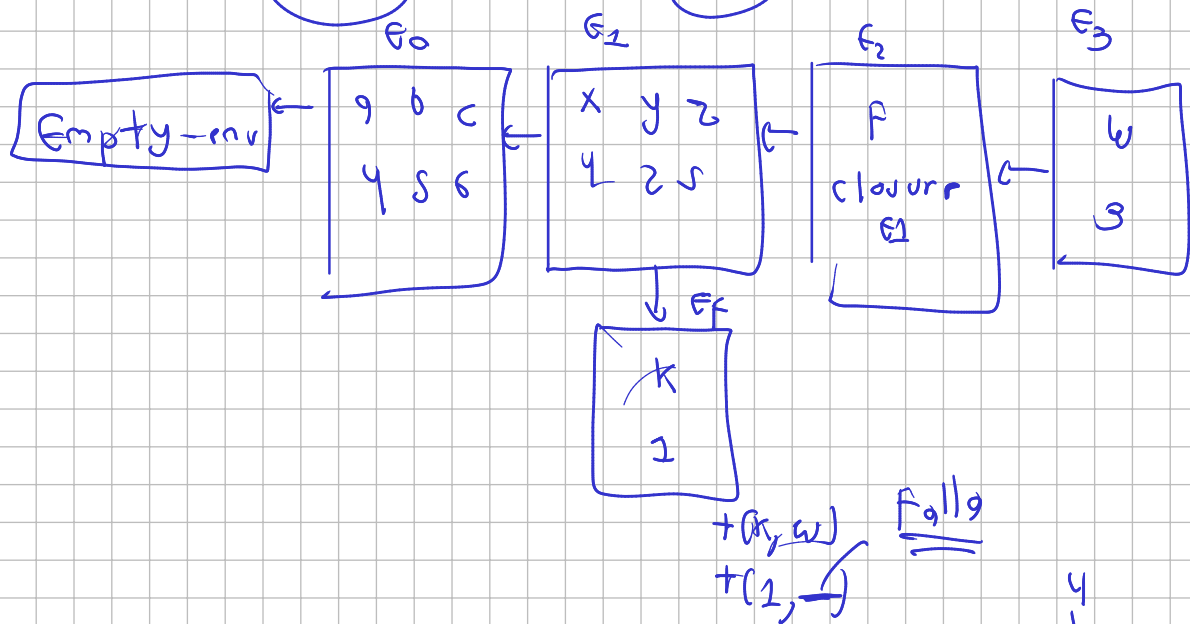
((g 5) 6)

```
#(struct:closure (z) #(struct:proc-exp #(struct:sum-prim) (#(struct:var-exp x)
#(struct:var-exp y) #(struct:var-exp z))) #(struct:ambiente-extendido (y) (6)
#(struct:ambiente-extendido (x) (5) #(struct:ambiente-extendido (x y z) (2 3 4))
```

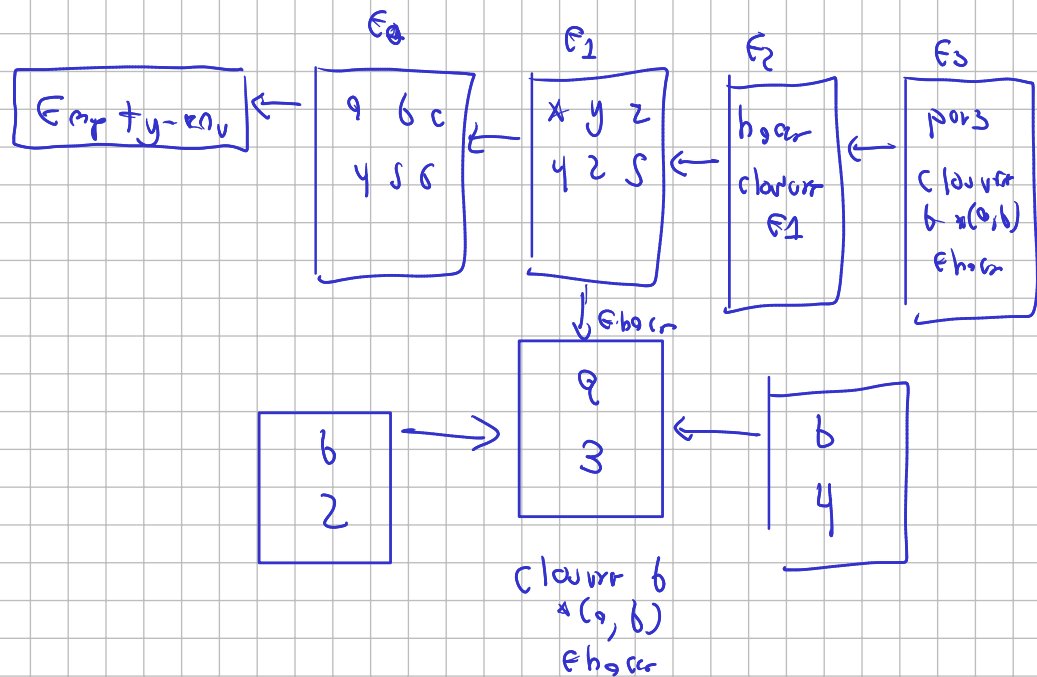
(— 7) + (x

+ (7 , 18) = 25

let f = proc(k) + (k,w) in let w = 3 in (f 1)

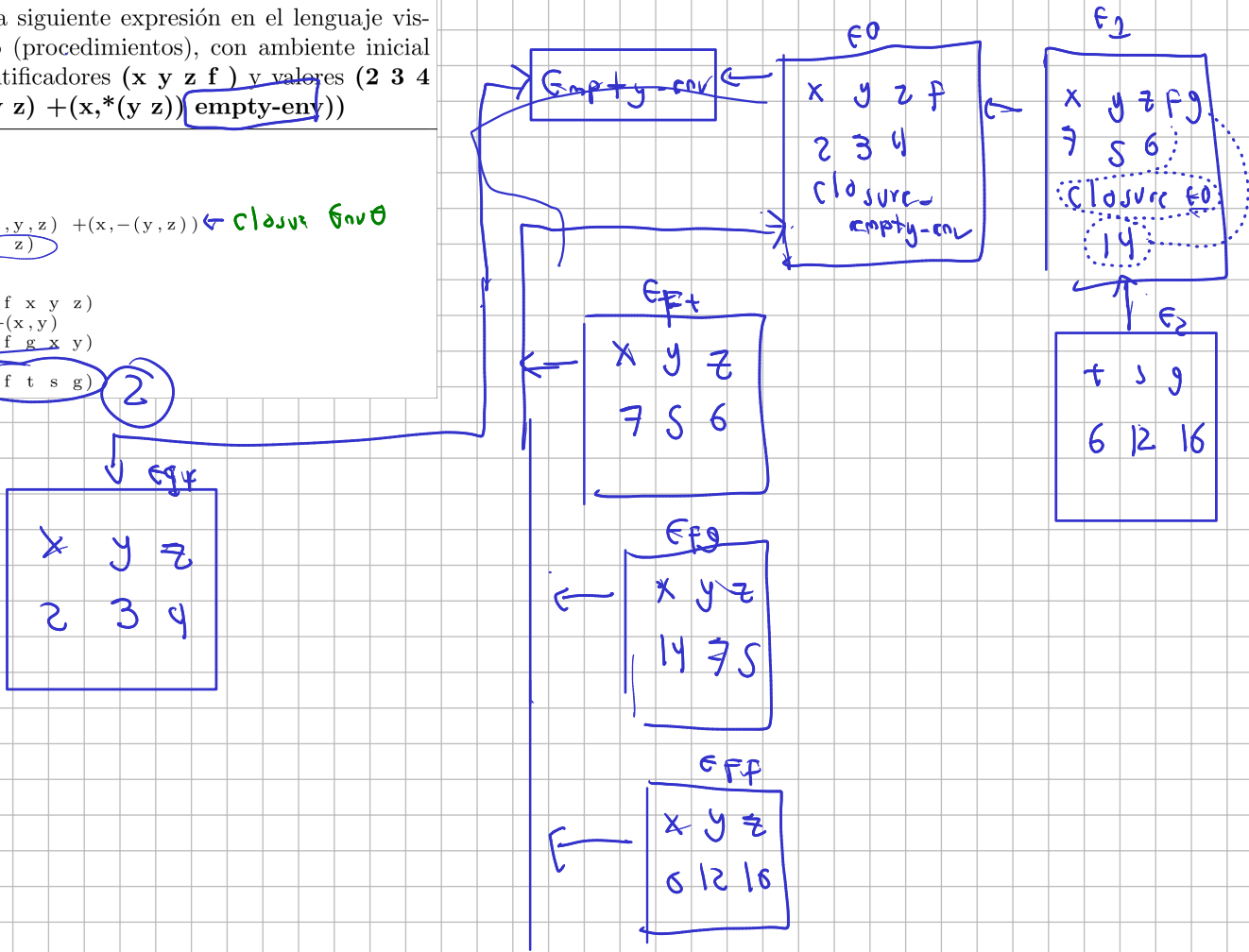


let hacer = proc(a) proc(b) *(a,b) in let por3 = (hacer 3) in +((por3 x), (por3 y))



Considere la siguiente expresi3n en el lenguaje visto en el curso (procedimientos), con ambiente inicial env0 con identificadores (x y z f) y valores (2 3 4 (closure'(x y z) +(x,*(y z)) empty-env))

```
let
  x = 7
  y = 5
  z = 6
  f = proc(x,y,z) +(x,-(y,z))
  g = (f x y z)
in
  let
    t = (f x y z)
    s = +(x,y)
    g = (f g x y)
  in
    (f t s g)
```



La declaraci3n proc(x) solo crea CLAUSURA (no mas)

La evaluaci3n (f x ...) saca el ambiente que esta en la clausura p y evalua el cuerpo en un ambiente extendido de p con los ids del proc y los valores que enviaron. ESTO ES ALCANCE ESTATICO.

El IF no genera ambientes, solo ramifica.

Toda expresi3n retorna un valor expresado: NUMEROS, BOOLEANOS o PROCVAL (clausuras)